

数据结构与算法

Data Structures

1

课程简介

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

- 课程简介
- 入门须知
- 课程要求
- 授课内容



目录

1	课程简介
2	入门须知
3	课程要求
4	授课内容
5	总结

课程简介

课程中文名称

数据结构与算法

课程英文名称

Data Structures and Algorithms

课程类型

软件工程专业基础课，必修

课程简介

课程目的

培养学生形成从思路到程序的能力。

培养学生用数据结构和算法解决问题的能力。

培养学生对编程和软件工程专业兴趣。

主讲教师

厦门大学信息学院软件工程系 黄炜 副教授

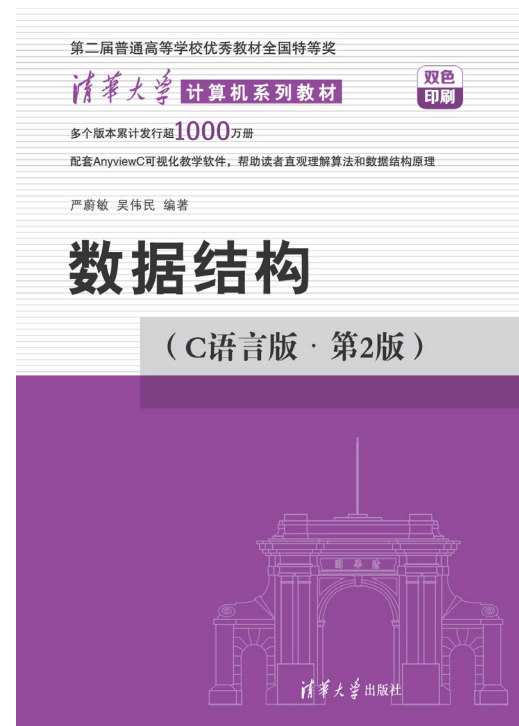
课程教材、参考书

官方教材

严蔚敏, 吴伟民, 数据结构 (C语言版), 清华大学出版社, 2021. (ISBN: 9000302001485)

课程类型

双语教学



课程教材、参考书

教学目的

- 熟练掌握数据结构的基本知识
- 掌握程序设计和实现的方法
- 掌握程序测试的方法
- 了解计算机相关专业术语

课程特色

阅读、测试

主讲教师介绍

- 黄炜，博士，厦门大学信息学院副教授
 - 2003年，厦门大学软件学院软件工程系，工学学士
 - 2007年，中国科学院大学，信息工程研究所，工学博士
 - 2013年，厦门大学，助理教授、副教授，硕士生导师
- 从事《C程序设计》教学已有10余年
- 研究方向：图像处理、人工智能、信息隐藏
- 联系方式
 - E-mail: whuang@xmu.edu.cn

课程定位

- 课程定位《数据结构与算法》是软件工程类专业的学科通修课程。也是研究生考试的必考科目。
 - 正规企业求职，或研究生复试，很可能有上机考试。
- 数据结构与算法不是一门语言，它是一种思维方式。
 - 讲授优化程序，使之运算更快、占用资源更少。
- 本课程以C语言为例，讲解数据结构。
 - 提供可以运行的代码（以课件为准）
- 硬件速度的提高，不是不重视算法性能的理由，而是我们追求高性能算法的动力。

应用需求

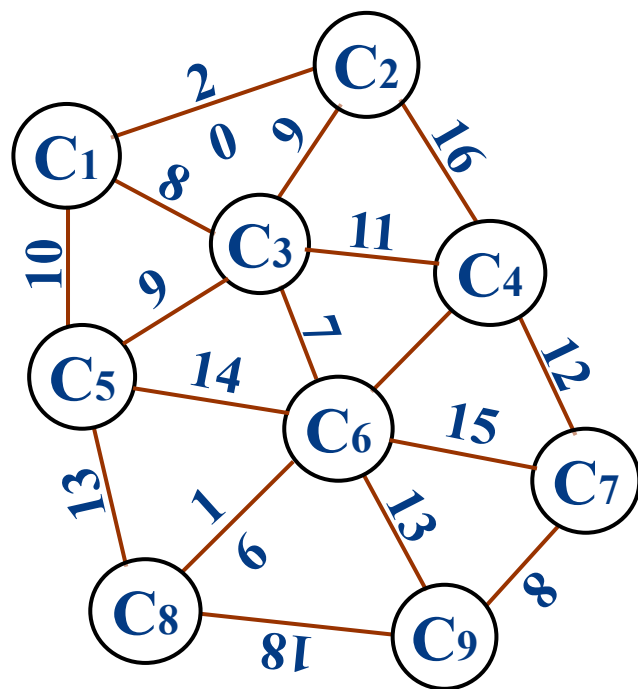
- 例1-7 最短路径问题

- 已知 n 个城市相互之间的距离。试找出从第1个城市出发到达第 j 个城市距离最短的路径。

- 例1-8 杨辉三角形

- 输出杨辉三角形($n \in [1, 12]$)。
- 如 $n=5$ 的输出结果：

1					
1	1				
1	2	1			
1	3	3	1		
1	4	6	4	1	
1	5	10	10	5	1



应用需求

第1种解答方法

```
Yanghui1(int n)
{ int a[N][N+1];
  printf("\n1\n\n1%6d\n\n", 1);
  for(i=1; i<n; ++i) {
    a[i-1][0]=a[i-1][i]=1;
    printf("1");
    for(j=1; j<=i; ++j) {
      a[i][j]=a[i-1][j-1]+a[i-1][j]; printf(a[i][j]);}
    printf("1\n");
  }
}
```

//时间复杂度 $O(n^2)$, 空间复杂度 $S(N^2)$

//第2种解答方法

```
Yanghui2(int n)
{ int a[2][N+1];
  printf("\n1\n\n1%6d\n\n", 1);
  for(i=1; i<n; ++i) {
    i0=(i-1)%2, i1=i%2;
    a[i0][0]=a[i0][i]=1;
    printf("1");
    for(j=1; j<=i; ++j)
    { a[i1][j]=a[i0][j-1]+a[i0][j];
      printf(a[i1][j]);}
    printf("1\n");
  }
}
```

// 时间复杂度 $O(n^2)$, 空间复杂度 $S(N)$

应用需求

//第3种解答方法

```
Yanghui3(int n)
{ printf("\n1\n\n1%6d\n\n", 1);
  for(i=2; i<=n; ++i) {
    c1=c2=1; printf("1");
    for(j=1; j<i; ++j)
      { c1*=(i-j+1); c2*=j;
        printf(c1/c2); }
    printf("1\n");
  }
} // 时间复杂度为 $O(n^2)$ , 空间复杂度为 $S(1)$ 
```

目录

1	课程简介
2	入门须知
3	课程要求
4	授课内容
5	总结

阁下在此

迷茫

我要去哪里 要一直坐着吗

我是谁 接下来干什么

我在哪

宇宙的边界在哪

人生的意义是什么 这个人是谁

我为什么会坐在这里

格物致知

幼儿时期

- 听爸爸妈妈的话

小学时期

- 爸爸妈妈说的不对
- 听老师的话

初中时期

- 爸爸妈妈和老师说的不对
- 模仿同学朋友

高中时期

- 爸爸妈妈老师和朋友说的不对
- 我是我，不一样的烟火

学习目标

- 早定目标

- 从事有意义的工作，个人价值、社会贡献
- 下一步继续深造攻读研究生？在IT企业求职？

- 规划路径

- 了解达到目标必须条件，提早努力
- 条件：GPA、学术竞赛获奖、研发项目、专业实习

- 随时调整

- 个人特质，周遭形势

各科的重要性

- 语文（表达）很重要
- 数学（算法）很重要
- 英语（语法）很重要
- 专业课很重要
- 其他课程也很重要（GPA）

程序设计与法律

- 民事

- 著作权纠纷；开发合同纠纷；泄密或竞业限制

- 刑事

- 第二百八十五条 非法侵入计算机信息系统罪

- 第二百八十六条 破坏计算机信息系统罪

- 第二百八十六条之一 拒不履行信息网络安全管理义务罪

- 第二百八十七条 利用计算机实施犯罪的提示性规定

- 第二百八十七条之一 非法利用信息网络罪

- 第二百八十七条之二 帮助信息网络犯罪活动罪

- 其它：共同犯罪：开设赌场罪

学习方法

- 如何学好数据结构与算法

- (1)掌握数据的抽象、组织和操作

- ✓ 表结构 ✓ 树结构 ✓ 图结构

- ✓ 查找 ✓ 排序

- (2)掌握数据在计算机中的表示、存储和算法实现。

- (3)通过典型应用，学会算法选择和性能优化，提高算法质量。

- (4)通过实验训练，提高计算思维和构造性思维能力，掌握问题的解决方法。

学习方法

- 数据结构与算法的学习过程，是进行程序设计的训练过程。学习数据结构，必须经过大量的实例训练，在实践中体会计算思维和构造性思维方法，掌握数据组织和程序设计技术。
 - 计算思维：是运用计算机科学的基础概念进行问题求解、系统设计、以及人类行为理解等涵盖计算机科学之广度的一系列思维活动。由美国CMU大学周以真教授于2006年3月首次提出。
 - 构造性思维：利用具体问题的典型特征，为待解问题设计一个合理的框架，从而使问题转化并得到解决。

如何学好这门课？

- 动手画图：遇到不懂的结构（比如二叉树遍历），别空想，拿出纸笔，画出来！
- 手写代码：别只看不练。核心算法（如链表增删、快排），合上书，自己手写一遍。
- 理解思想：别死记硬背代码。搞懂“为什么”要这么设计（比如为什么要有循环链表？），比记住代码更重要。
- 善用调试：程序跑不通？学会用调试器（Debugger）一步步看变量的变化，这是程序员的必备技能。

目录

1	课程简介
2	入门须知
3	课程要求
4	授课内容
5	总结

课程要求

• 理论课

- 适当提前到课，着装得体，不迟到缺勤
- 电子设备静音，不打闹，不做与课堂无关的事
- 带上**笔记簿和笔**，做好笔记，下课凭笔记照片积分
- 积极思考，踊跃回答

• 实验课

- 禁止在机房用餐，不做与课堂无关的事
- 下课后电脑关机，桌椅和设备恢复原状

课程要求

- 出勤要求

- 旷课按次扣分

- 事前请假可通过QQ私信，事后请假应办理请假手续

- 按校规，理论课或实验课缺勤（含请假）达1/3者，记0分

- 课后要求

- 鼓励上机练习编程，不在练习出错，必在工作出错。

- 不要害怕软件错误，只要不是物理损坏，都可以恢复。

- 多动笔，多动脑，多思考，多提问。

- 阅读很重要，只写不读，必然导致土话连篇。

- 国内外很多很好的代码供大家阅读学习。

作业要求

- 理论作业

- 课程提供习题集，每节课布置作业
- 自授课当日为第一日，第七日结束前提交

有同学问：老师我
不知道作业是什么？

- 实验作业

- 在拼题 A 网（<https://pintia.cn/>），先注册，有八次实验
- 截止时间由实验老师确定

- 按时完成，认真作答，鼓励展开讨论

- 严禁作弊，不得抄袭或协助他人抄袭作业
- 合理使用AIGC工具，不要代劳主要工作（期末有上机考）

作业要求

- 作业的批改采取登记制
 - 正常提交，不乱写，少量错误，100分。
 - 认真写且补交：60分。
 - AI代写、乱写、缺交，一经发现，0分。
- 学有余力的同学：外网在线测评系统做题
 - 按NOIP的要求学习算法
 - 上OJ系统（洛谷、力扣等）寻找题目，先易后难
 - 参加ICPC等程序设计竞赛

期末成绩构成

- 期末笔试 (40%)
 - 统一闭卷笔试
- 平时成绩 (60%)
 - 期中考试 (10%)
 - 期末上机考试 (20%)
 - 理论作业 (16%)
 - 实验作业 (24%)
 - 考勤：倒扣分
 - 迟到5分钟以上扣0.5分，迟到23分钟以上扣1分

目录

1	课程简介
2	入门须知
3	课程要求
4	授课内容
5	总结

研究数据表示

- 程序 = 数据结构 + 算法

例：位图，一般采用二维数组存储其灰度（亮度）信息。为什么？

- 数据结构：程序以何种形式来存储信息

- 示例：

- 题：某影评机构请你做一个程序，输入客户看过的电影列表，记录片名、（中间省略若干字）、您的评价等。

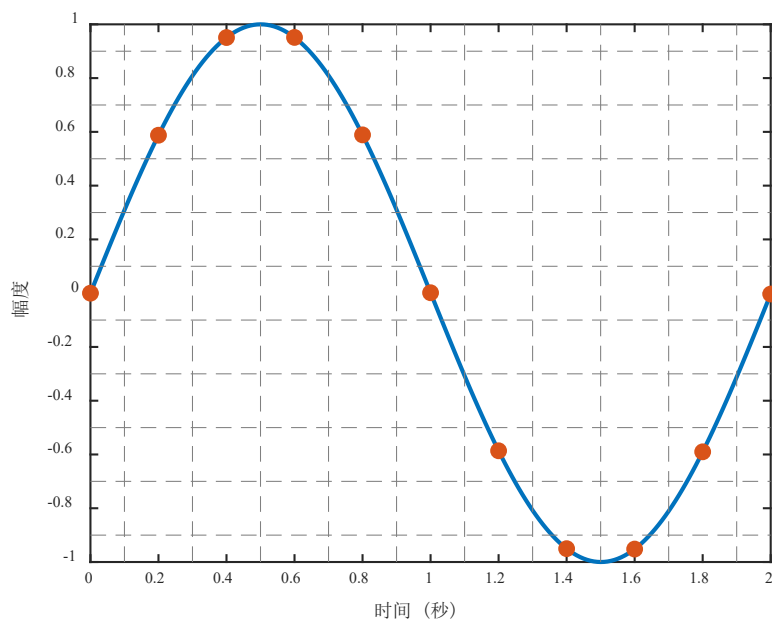
- 拟采用数据结构

- 结构体：意义不同，数据类型不同，范畴相近
 - 数组：意义相同，数据类型相同，仅有次序区别

基本概念

- 数据 (Data) : 万物皆可输入

- 将自然界的模拟信号转换成数字形式的过程称为数字化。
- 脉冲编码调制采样间隔 $125\mu\text{s}$ ，每个样本用0~255的整数表示



$(0,0), (0.2,0.6), (0.4,1), (0.6,1), (0.8,0.6),$
 $(1,0), (1.2,-0.6), (1.4,-1), (1.6,-1),$
 $(1.8,-0.6), (2,0)$

```
int A = [0, 3, 5, 5, 3, 0, -3,  
-5, -5, -3, 0];
```

基本概念

- 数据（Data）：万物皆可输入
 - 数据是对客观事物的符号表示，是所有能够输入计算机并被计算机程序处理的符号的总称。
- 数据元素（Data Element）：打包处理的整体
 - 数据元素是数据的基本单位。
 - 在计算机程序中通常作为一个整体进行考虑和处理。
 - 提到《走向共和》的时候，已经包括片名、导演、评分等影评整体。
- 数据对象（Data Object）：强调“同类元素的集合”
 - 数据对象是性质相同的数据元素集合，是数据的一个子集。
 - 把张三、李四和王五等提交的影评记录打包成整体，称数据对象。

基本概念

- 具有相同性质的数据元素的集合。如
 - (1)英文字母数据对象
 - $C=\{‘A’, ‘B’, ……., ‘Z’, ‘a’, ‘b’, ……., ‘z’\}$ 。
 - (2)自然数数据对象 $N=\{1, 2, …….\}$ 。
 - (3)书目信息数据对象：包含书号, 书名, 作者, 出版社, 出版日期等数据项的数据元素集。

基本概念

- 利用数组解决问题的局限性：不可扩充
 - 虽然电影名总长度可调研，预先设定数组长度
 - 电影数量不可预设，用户也没法算一辈子会看几部电影
- 改用结构体链表解决问题，但仍存在局限性
 - 编码细节（链表增删）与概念模型（影评业务）混合了
 - 如果你想做一个电话簿管理软件，仅做小修改是不够的
 - 如果某天我发现了影评编码的错误，很难平移到电话簿

基本概念

- C语言中没有合适影评和电话簿的数据类型
 - 我们设计了一个结构体，再变成链表，来表示影评。
 - 我们设计了一个新结构体，再变成链表，来表示电话簿。
- 系统性的做法：定义类型（Type）
 - 数据类型（Data type）是一个值的集合和定义在这个值集上的一组操作的总称。
 - 操作是核心部分，没有操作的数据类型是没有什么用的
 - 设想：一个不能加减乘除的int类型，是没有用的
 - 数学提供了整数的抽象概念，C提供了概念的实现
 - 但是int类型并没有很好地实现整数，整数是无穷的

基本概念

- 定义一个新的类型的成功方法
 - 为类型的属性和可对类型执行的操作提供一个抽象的描述
 - 开发一个实现该ADT的编程接口，编写代码来实现接口
- 抽象数据类型（Abstract Data Type，ADT）
 - ADT是指一个数学模型以及定义在该模型上的一组操作。
 - ADT定义取决于数据类型的数学（逻辑）特性，与在计算机内部表示和实现无关。
 - 抽象数据类型和数据类型实质上相同。

类型

如：int类型

属性集

整数值、正负

加减乘除模、比较.....

操作集

核心部分

基本概念

- 数据结构（ Data Structure ）
 - 数据结构研究数据元素之间的相互关系，以及定义在其上的一组基本操作。
- 数据结构主要包括三部分的内容
 - 逻辑结构：描述数据元素之间的逻辑关系
 - 如：线性结构？树形结构？圆形结构？
 - 存储结构：逻辑结构在计算机中的物理实现
 - 如：顺序存储（数组）？链式存储（链表）？索引？散列？
 - 运算集合：实现对数据元素及其逻辑关系的基本操作
 - 如：插入、删除、查找、输出等。

例题

例 1-2 “成绩表”的存储结构描述：

```
typedef struct
{
    char No[14];    // 学号
    char Name[20];  // 姓名
    int Attend[2];  // 出勤
    int Homework[3]; // 平时成绩
    .....
} Elem; //数据类型描述
```

用顺序表描述逻辑关系 **Elem Info[N];**;
还可以用链式存储方式描述逻辑关系。

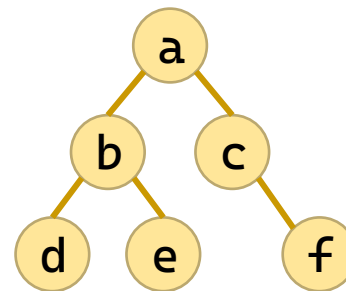
例题

- 数据结构的形式定义： $\text{Data_Structure} = (D, R)$
 - D : 数据对象， R : 定义在 D 上的关系集。

例1-3 数据结构 $DS=(D, R)$ ，其中， $D=\{a_1, a_2, a_3, a_4, a_5, a_6\}$ ； $R=\{<a_1, a_2>, <a_2, a_3>, <a_4, a_5>, <a_5, a_6>, <a_1, a_4>, <a_2, a_5>, <a_3, a_6>\}$ 。

a_1	a_2	a_3
a_4	a_5	a_6

例1-4 数据结构 $DS=(D, R)$ ，其中， $D=\{a, b, c, d, e, f\}$
 $R=\{<a, b>, <a, c>, <b, d>, <b, e>, <c, f>\}$



基本概念

- ADT 的两个重要特征：
 - 数据抽象性：用ADT描述程序的实体时，强调的是其本质特征、所能完成的功能，以及它和外部用户的接口。
 - 数据封装性：将实体外部特性和内部实现细节分离，并对外部用户隐藏内部实现细节。
- 抽象数据类型的形式定义是一个三元组
 - $ADT = (D, R, P)$
 - 其中，D是数据对象，R是D上的关系集，P是对D和R的基本操作集。

基本概念

- 抽象数据类型的定义格式：

ADT 抽象数据类型名 {

 数据对象：<数据对象的定义>

 数据关系：<数据关系的定义>

 基本操作：<基本操作的定义>

} ADT 抽象数据类型名

- 其中，数据对象和数据关系的定义一般采用伪码描述。

- 基本操作的定义格式如下：

- 基本操作名：(参数表)
- 初始条件：<描述操作执行之前，数据结构和参数应满足的条件>
- 操作结果：<说明操作正常结束之后，数据结构的变化情况和应返回的结果>

示例

例1-5 抽象数据类型复数的定义。

ADT Complex {

数据对象: $D = \{e_1, e_2 \mid e_1, e_2 \in \text{RealSet}\}$

数据关系: $R = \{r_1\}$

$r_1 = \{ \langle e_1, e_2 \rangle \mid e_1 \text{表示复数的实数部分, } e_2 \text{是复数的虚数部分} \}$

基本操作:

InitComplex (&Z, v1, v2)

操作结果: 构造复数Z, 实部和虚部分别被赋以参数v1和v2的值。

DestroyComplex (&Z)

操作结果: 销毁复数Z。

GetReal (Z, &zr)

初始条件: 复数已存在。

操作结果: 用zr返回复数Z的实部值。

GetImag (Z, &zi)

初始条件: 复数已存在。

操作结果: 用zi返回复数Z的虚部值。

Add (Z1, Z2, &sum)

初始条件: Z1, Z2是复数。

操作结果: 用sum返回Z1与Z2的和值。

} ADT Complex

基本概念

- 算法 (Algorithm)
 - 算法是求解特定问题的过程描述、操作步骤或指令序列。
- 数据结构和算法的关系
 - 好的数据结构可以提高算法性能；
 - 通过算法研究可以加深理解数据结构。
- 算法的5个重要特性
 - 有穷性 (Finiteness)：对于任意合法输入，一个算法必须总在执行有穷步之后结束，且每一步可在有穷时间内完成。
 - 算法必须是有穷的，而程序可以是无穷的

基本概念

- 算法的5个重要特性

- 确定性 (Definiteness) : 算法中每条指令必须有确切的含义, 相同输入经过相同的执行路径, 得到相同输出。
- 可行性 (Effectiveness) : 算法中描述的操作都可以通过已经实现的基本运算执行有限次来实现。
- 输入项 (Input) : 一个算法有零个或多个输入, 这些输入取自于某个特定的对象的集合。
- 输出项 (Output) : 一个算法有一个或多个输出, 这些输出是与输入有着某种特定关系的量。

基本概念

- 算法设计的要求

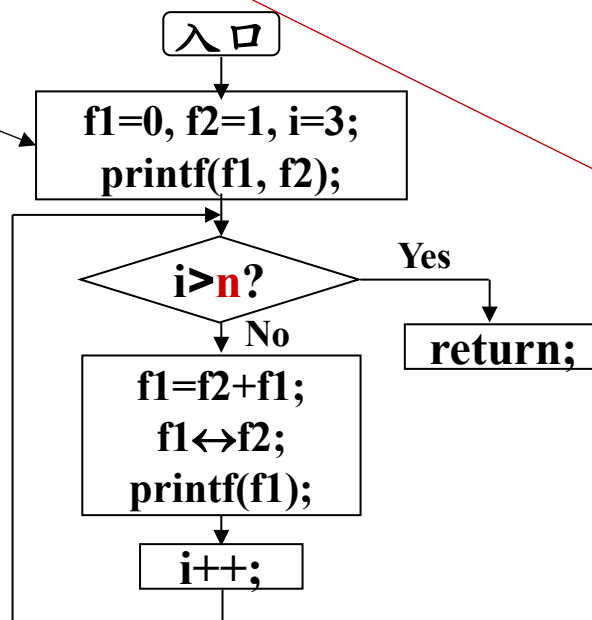
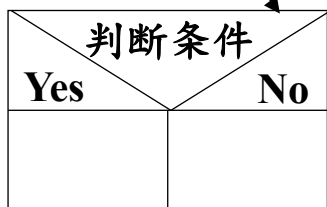
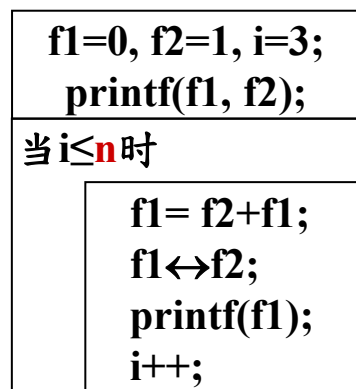
- 正确性：算法应能够正确地解决求解问题。
- 可读性：算法应具有良好的可读性，以帮助人们理解。
- 健壮性：输入非法数据时，算法能适当地做出反应或进行处理，而不会产生莫名其妙的输出结果。
- 高效率：效率是指算法的执行时间。力求设计一个高效率的解决特定问题的算法。
- 低存储：存储量是指算法执行过程中所需的最大存储空间。算法设计应考虑尽可能低的空间开销问题。

基本概念

• 算法描述方法

- 类C语言描述
- 数学方法描述
- 流程图
- N-S图描述
- 自然语言描述

```
Yanghui(int n) {  
    for (i = 2; i ≤ n; ++i) {  
        m1 = m2 = 1;  
        for (j = 1; j < i; ++j) {  
            m1 = m1 * (i - j + 1);  
            m2 = m2 * j;  
            printf(m1 / m2);  
        }  
    }  
}
```



- ① f1=0, f2=1, i=3 ;
- ② 输出 f1, f2 ;
- ③ if (i>n) return ;
- ④ f1=f2+f1 ;
- ⑤ 交换 f1 和 f2 的值 ;
- ⑥ 输出 f1 ;
- ⑦ i++ ;
- ⑧ go ③ ;

算法复杂度

- 算法复杂度是算法占用机器资源的量
 - 通常表示为，与问题规模 n 之间的关系
- 时间复杂度(Time Complexity) $T(n)=O(f(n))$
 - 算法运行所需要的执行时间度量。问题规模 n 之间的关系。
 - 最坏时间复杂度：最坏情况下
 - 平均时间复杂度：所有可能输入等概率情况下
 - 最好时间复杂度：最好情况下
- 空间复杂度(Space Complexity) $S(n)=O(f(n))$
 - 算法运行所需要的存储空间度量。

基本概念

- 时间复杂度的计算

- 语句频度

- 第2、7行1次
 - 第3行 $n+1$ 次
 - 第4、5行 n 次

```
1. void loveDSA(int n) {  
2.     int i = 1;  
3.     while (i <= n) {  
4.         i++;  
5.         printf("I love DSA %d\n", i);  
6.     }  
7.     printf("I love DSA more than %d\n", n);  
8. }
```

- 时间开销与问题规模 n 的关系： $T(n)=3n+3$

- 复杂的程序精确计算 $T(n)$ 是困难的

- 关注于数量级，可以简化问题 $T(n)=3n+3=O(n)$

- 用 $O(\cdot)$ 表示同阶，即：当 $n \rightarrow \infty$ 时，二者之比为常数

基本概念

• 规则

- 多项相加，只保留最高阶的项，且系数变为1

$$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$$

- 多项相乘，都保留

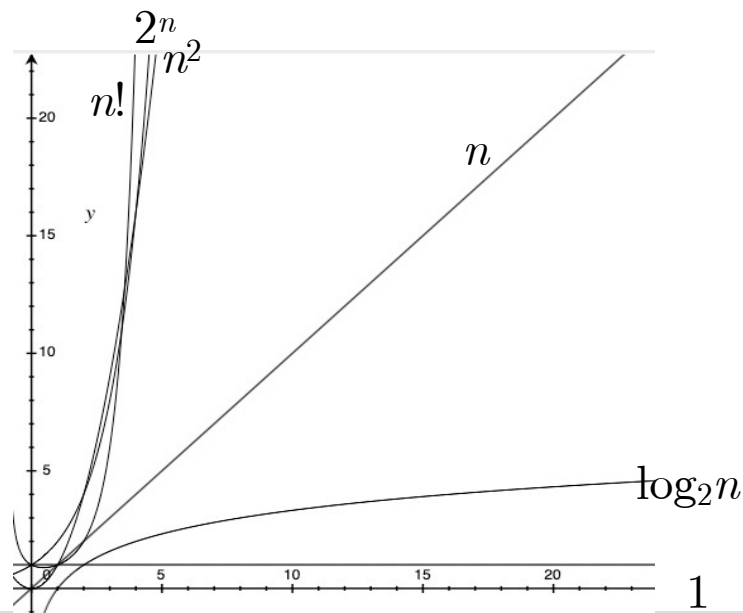
$$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$$

$$O(1) < O(\log_2 n) < O(n)$$

$$< O(n \log_2 n) < O(n^2) < O(n^3)$$

$$< O(2^n) < O(n!) < O(n^n)$$

算法的性能问题只有在 n 很大时才会暴露出来。



基本概念

- 空间复杂度的计算

- 无论问题规模怎么变，算法运行所需的内存空间都是固定的常量，算法空间复杂度为 $S(n) = O(1)$
- 多维数组带来的复杂度开销
- 递归带来的复杂度开销：与递归调用的深度相关

```
1. int Search(int A[], int n, int k) {  
2.     int i = n - 1;  
3.     while (i >= 0 && A[i] != k)  
4.         i--;  
5.     return i;  
6. }
```

基本概念

- 算法时间复杂度的估算方法

- 原操作：对于所研究问题的基本操作

- 它是算法中重复执行次数最多、且与问题规模 n 直接相关的那个操作。通常是循环体最深处的语句。

- 从算法中选取一种原操作，将该操作重复执行的次数作为算法时间复杂度的衡量准则。

- 时间复杂度与原操作的执行次数之和成正比。

- 例 `for (i=1; i<=n; ++i) s+=i;` $\Rightarrow T(n)=O(n)$

基本概念

例1-5 分析下列算法的时间复杂度。

```
F(int a[ ],int n)
{ for(i=0;i<n-1;++i)
  { j=i;
    for(k=i+1;k<n;++k )
      if(a[k]<a[j]) j=k; //原操作
      a[j]↔a[i] //交换a[j]和
a[i]
  }
}
```

算法的时间复杂度为 $O(n^2)$

```
F(int A[ ],int l,int r)
{ while(l<r)
  { while(A[r]≥0) --r;
    A[l]=A[r];
    while(A[l]≤0) ++l;
    A[r]=A[l];
  }
}
```

算法的时间复杂度为 $O(n)$

课程知识安排

2

线性表

3

栈和队列

4

串

5

数组和广义表

6

树和二叉树

7

图

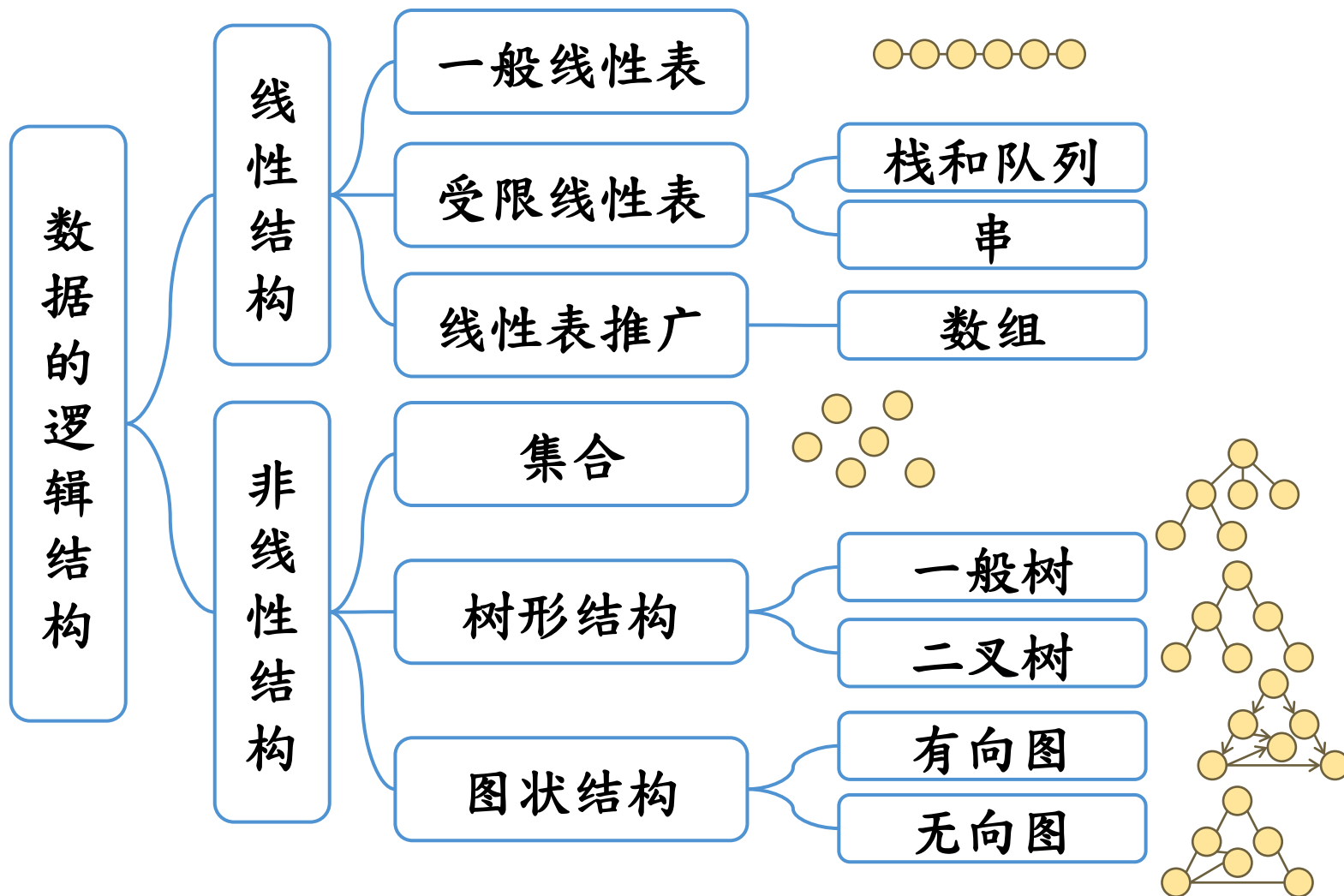
9

查找

10

内部排序

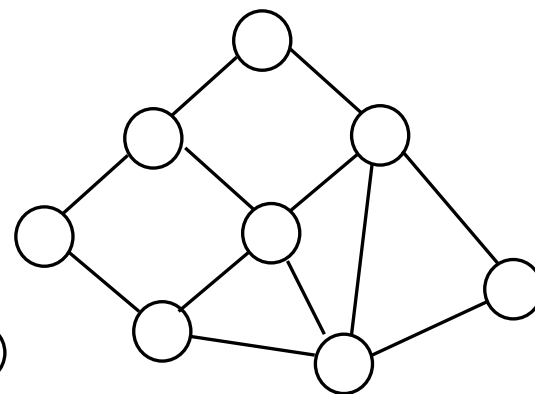
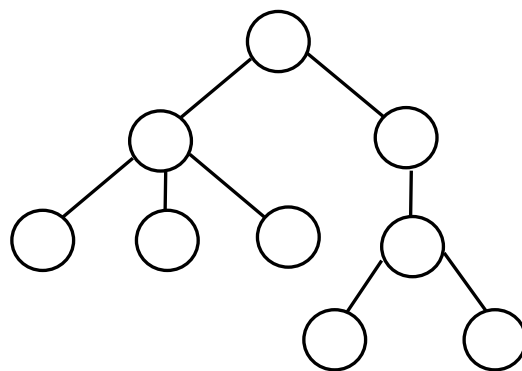
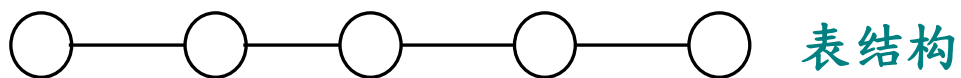
主要内容



主要内容

- 数据结构与算法学什么？

- 表结构
- 树结构
- 图结构
- 查找、排序



主要内容

- 线性表
 - 顺序表：线性表；三元组顺序表
 - 链表：循环链表；双向链表
- 栈；队列；循环队列
 - 迷宫问题；括号匹配检验；算术表达式求值；递归算法
- 树的基本概念；二叉树的性质和存储结构
 - 二叉树的遍历方法(先序、中序、后序)
 - 线索二叉树；哈夫曼树；哈夫曼编码
 - 树的存储结构；子集树；排列树

主要内容

- 图的基本概念、存储结构与遍历
 - 最小生成树；Prim算法；Kruskal算法
 - 最短路径；Dijkstra算法；Floyd算法
 - 有向无环图；拓扑排序；关键路径
- 查找
 - 静态查找表
 - 动态查找表：二叉排序树；平衡二叉树
 - 散列表基本概念；哈希函数；处理冲突的方法

主要内容

- 排序

- 简单排序：插入排序；冒泡排序；shell排序
- 归并排序；快速排序；堆排序；计数排序
- 树形选择排序；基数排序；桶排序

目录

	1	课程简介
	2	入门须知
	3	课程要求
	4	授课内容
	5	总结

内容要点

- 课程简介
- 入门须知
- 课程要求
- 授课内容



数据结构与算法

Data Structures

1

谢谢观看

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang